

Project 2 Part 2

1 - Model building and interpretation.

- a - Build various models (You can choose to build models for either or all of descriptive, predictive or prescriptive purposes)
- b - Test your predictive model against the test set using various appropriate performance metrics
- c - Interpretation of the models

2 - Model Tuning and business implication

- a - Ensemble modelling, wherever applicable
- b - Any other model tuning measures (if applicable)
- c - Interpretation of the most optimum model and its implication on the business

1 - Descriptive Model:

Before creating model, we have performed 2 tests.

 Bartlett's Test of Sphericity

 KMO Test

1 - Bartlett's Test of Sphericity – Before Test of Sphericity to check correlations between variables.

Bartlett's test of sphericity tests the hypothesis that the variables are uncorrelated in the population.

HO: All variables in the data are uncorrelated

HA: At least one pair of variables in the data are correlated

If the null hypothesis cannot be rejected, then PCA is not advisable.

Note - Reject HO if p-value < 0.05

After performing the test, we have received the p-value is 0.0. We reject the Null hypothesis. Hence, we can perform the PCA.

2 - KMO Test - To the correlation we have performed one more test which is The Kaiser-Meyer-Olkin (KMO) Test.

KMO Value | Interpretation | PCA Suitability

0.90 to 1.00 | Marvelous: Strong patterns of correlation exist. | Highly suitable for PCA.

0.80 to 0.89 | Meritorious: Variables are adequately correlated. | Suitable for PCA.

0.70 to 0.79 | Middling: Moderate correlations are present. | PCA can be considered.

0.60 to 0.69 | Mediocre: Weak correlations exist. | PCA is questionable but possible.

0.50 to 0.59 | Miserable: Very weak correlations. | PCA is likely not suitable.

Below 0.50 | Unacceptable: Data is poorly correlated. | PCA should not be performed.

AS per details value of KMO_model is 0.6366086675026923. Hence, we can't go for PCA. After the test we can observed that correlation exists in this condition PCA is questionable but possible.

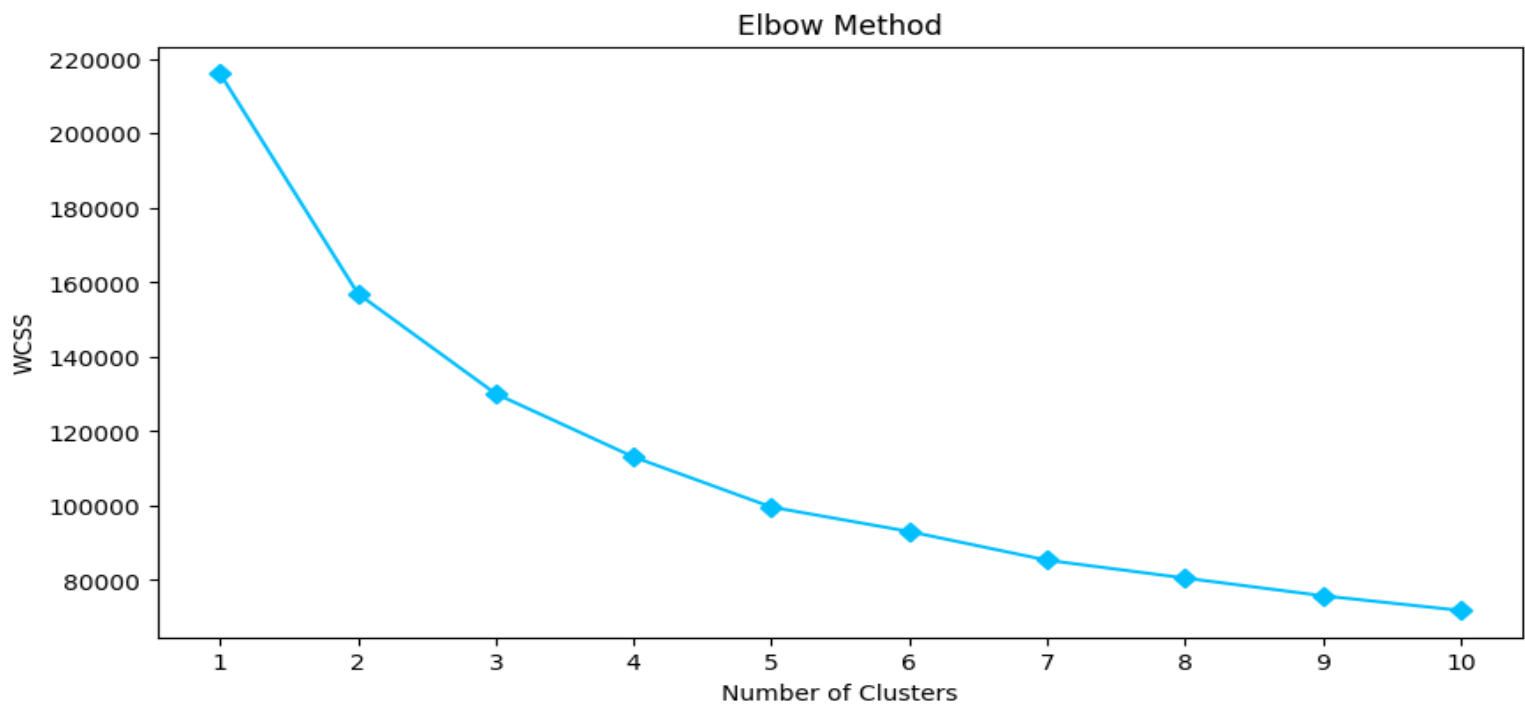
Hence, we have decided to not to performed PCA.

K-Means Clustering - A partitioning method that divides data into a pre-defined number of clusters (k). It is Partition-based clustering. Each data point belongs to exactly one cluster. where **n** is the number of points, **k** is the number of clusters, and **t** is the number of iterations. It is Suitable for large datasets.

Calculating WSS for other values of K - Elbow Method

[216130.0,
156988.62,
130088.36,
113103.63,
99632.45,
93065.77,
85356.49,
80559.58,
75806.24,

71856.07]



Check the silhouette scores against n_clusters

n_clusters = 2, silhouette_score = 0.2782

n_clusters = 3, silhouette_score = 0.2405

n_clusters = 4, silhouette_score = 0.2455

n_clusters = 5, silhouette_score = 0.2561

n_clusters = 6, silhouette_score = 0.2450

n_clusters = 7, silhouette_score = 0.2078

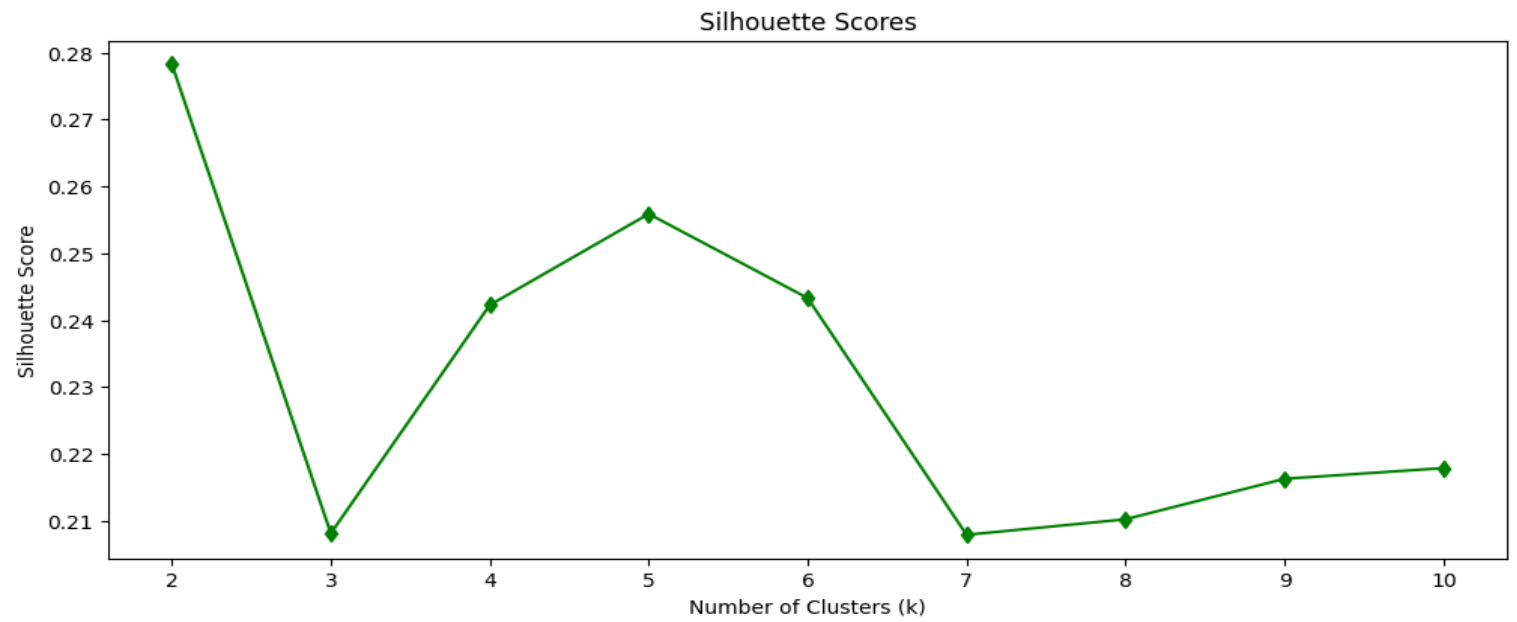
n_clusters = 8, silhouette_score = 0.2167

n_clusters = 9, silhouette_score = 0.2163

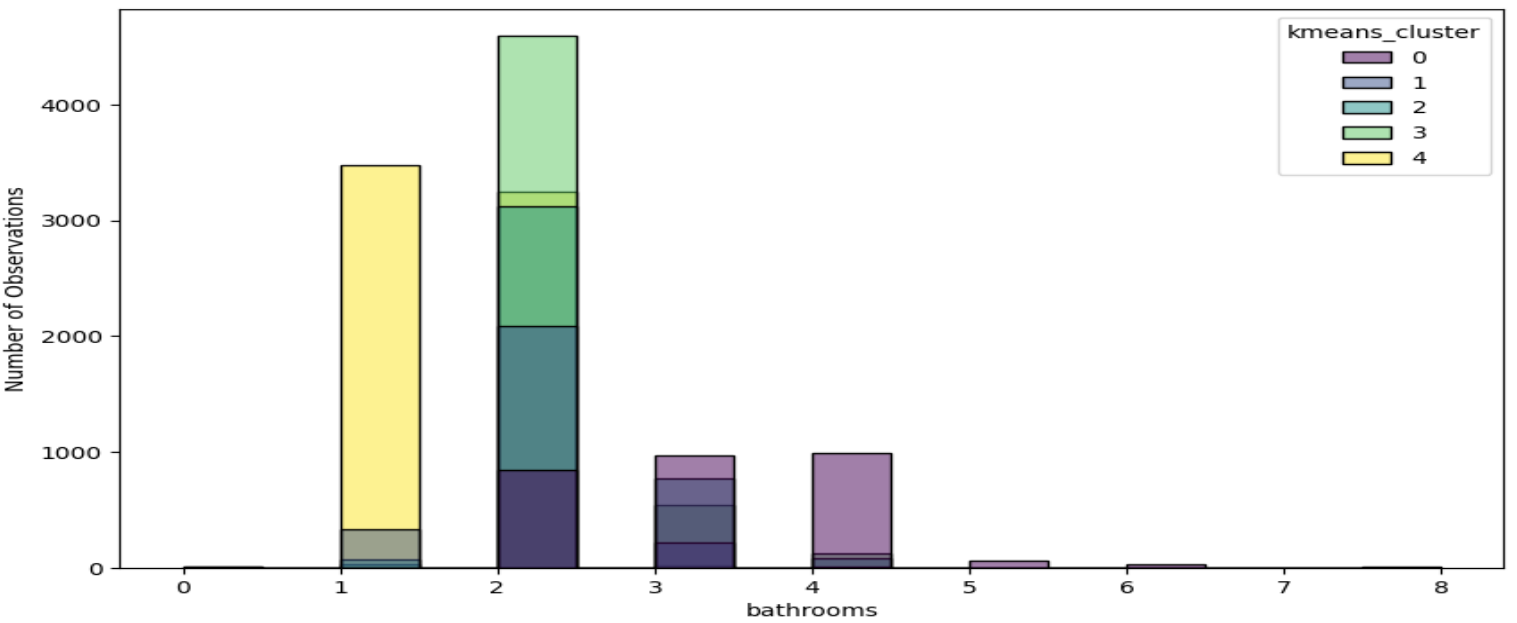
n_clusters = 10, silhouette_score = 0.2188

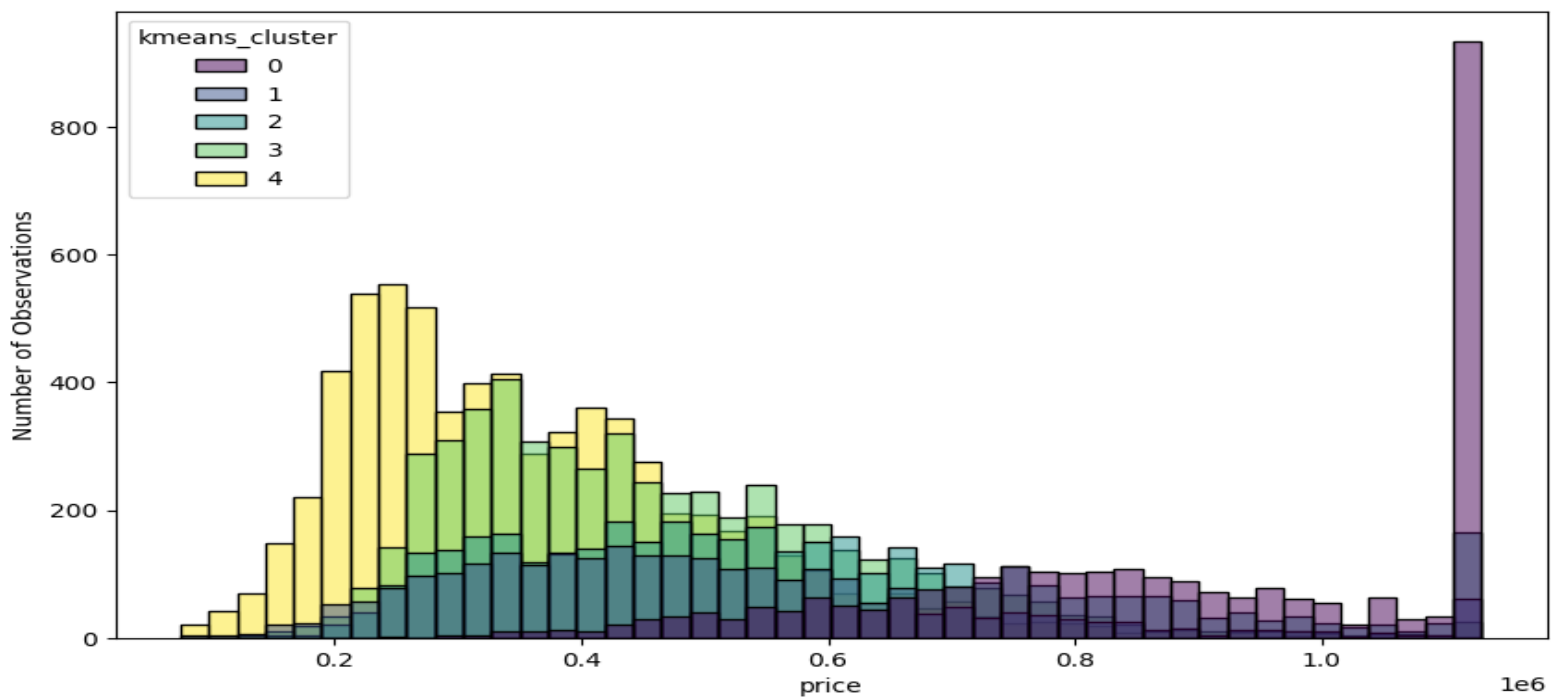
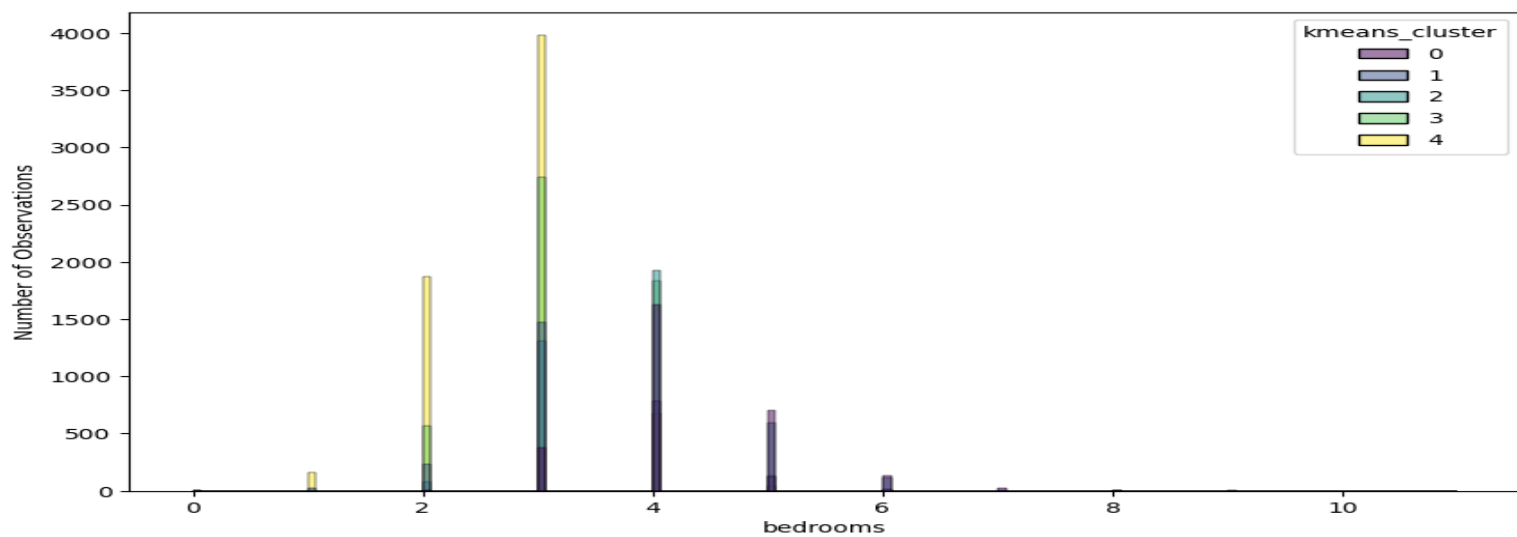
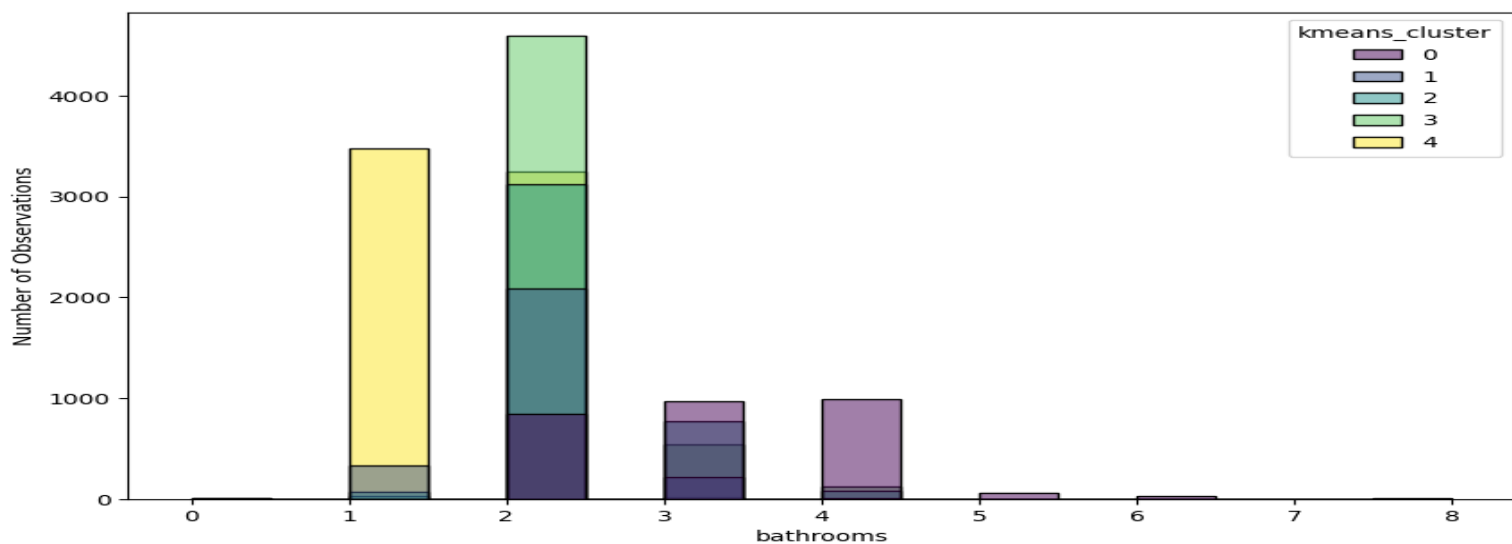
Determine the optimal number of clusters using the elbow method and silhouette score -

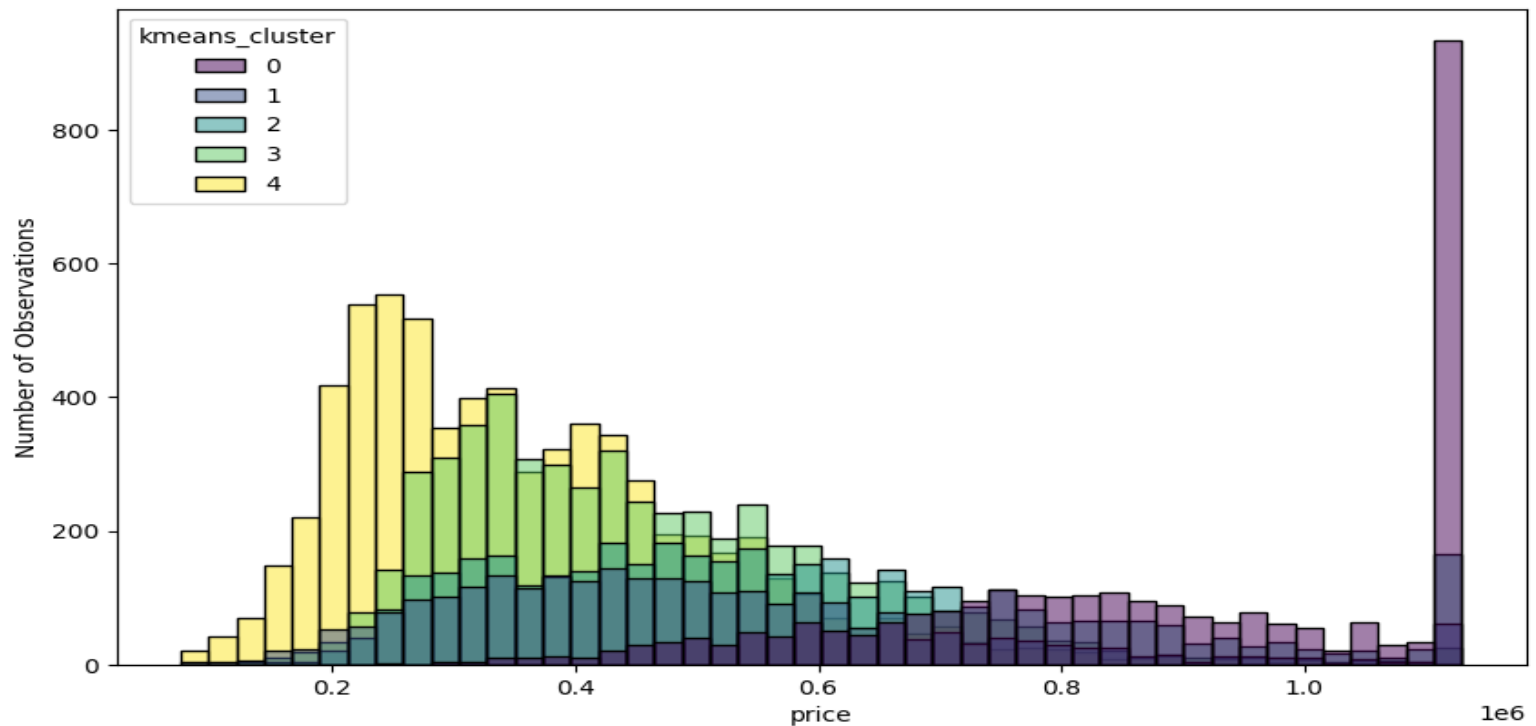
Observations: We can see from the plot that silhouette score is highest for k=5. Well, that makes it slightly easy for us and we can start with first understanding these 5 clusters. Final clusters will be 5. Append Clusters to the original dataset



	price	bedrooms	bathrooms	living_measure	lot_measure	floors	ceil_measure	basement	total_area	Age_of_house	kmeans_cluster
0	600000.0	4	2	3050.0	9440.0	1	1800.0	1250.0	12490.0	49	2
1	190000.0	2	1	670.0	3101.0	1	670.0	0.0	3771.0	67	4
2	735000.0	4	3	3040.0	2415.0	2	3040.0	0.0	5455.0	48	3
3	257000.0	3	2	1740.0	3721.0	2	1740.0	0.0	5461.0	5	3
4	450000.0	2	1	1120.0	4590.0	1	1120.0	0.0	5710.0	91	4







Cluster Analysis and Recommendations

Cluster 0

- Average Price: \$887,388.80
- Bedrooms: 4.25
- Bathrooms: 3.13
- Living Measure (sq. ft.): 3,533.82
- Lot Measure (sq. ft.): 12,080.62
- Floors: 1.92
- Ceil Measure: 3,127.64
- Basement (sq. ft.): 388.83
- Total Area (sq. ft.): 15,418.79
- Age of House (years): 24.76

Insights:

This cluster represents large and relatively new houses with high living and lot areas, and higher-than-average prices.

Houses in this cluster are premium properties, likely appealing to affluent buyers seeking spacious homes.

Recommendations:

Target luxury buyers for these properties through high-end marketing channels.

Highlight features like spacious living areas and basements in advertisements.

Cluster 1

- Average Price: \$493,740.72
- Bedrooms: 3.30
- Bathrooms: 1.96
- Living Measure (sq. ft.): 2,127.64
- Lot Measure (sq. ft.): 17,706.67
- Floors: 1.27
- Ceil Measure: 1,849.01
- Basement (sq. ft.): 273.98
- Total Area (sq. ft.): 20,202.80
- Age of House (years): 42.78

Insights:

These homes are mid-sized houses on large lots, priced in the mid-range.

The houses are relatively older, but the large lot sizes may offer opportunities for redevelopment or renovations.

Recommendations:

Market these properties to families who value outdoor space and are open to renovations.

Highlight the potential for adding value to the property by upgrading or modernizing.

Cluster 2

- Average Price: \$574,462.93
- Bedrooms: 3.86
- Bathrooms: 2.21
- Living Measure (sq. ft.): 2,276.63
- Lot Measure (sq. ft.): 7,617.06
- Floors: 1.09
- Ceil Measure: 1,456.93
- Basement (sq. ft.): 811.26
- Total Area (sq. ft.): 9,893.74
- Age of House (years): 59.90

Insights:

This cluster represents older houses with mid-range prices and moderate living areas.

The relatively large basements could appeal to buyers looking for additional usable space.

Recommendations:

Market these properties to buyers looking for homes with character or investors seeking to modernize older houses.

Emphasize the functional basements and potential for customization.

Cluster 3

- Average Price: \$459,304.00
- Bedrooms: 3.29
- Bathrooms: 2.14
- Living Measure (sq. ft.): 2,008.34
- Lot Measure (sq. ft.): 5,098.94

- Floors: 2.08
- Ceil Measure: 1,949.80
- Basement (sq. ft.): 58.40
- Total Area (sq. ft.): 7,107.28
- Age of House (years): 15.67

Insights:

These houses are relatively new and compact homes, with moderate prices. Small lot sizes and minimal basement space indicate these are likely urban or suburban homes.

Recommendations:

Market to young professionals or small families who prioritize modern homes in convenient locations.

Emphasize the newer construction and lower maintenance requirements.

Cluster 4

- Average Price: \$360,931.50
- Bedrooms: 2.79
- Bathrooms: 1.48
- Living Measure (sq. ft.): 1,306.22
- Lot Measure (sq. ft.): 7,202.45
- Floors: 1.03
- Ceil Measure: 1,201.96
- Basement (sq. ft.): 104.26
- Total Area (sq. ft.): 8,509.91
- Age of House (years): 63.14

Insights: These are small, older homes with lower prices, likely appealing to budget-conscious buyers.

Modest living areas and minimal basements suggest compact homes.

Recommendations:

Market to first-time homebuyers, retirees, or investors seeking affordable properties.

Highlight affordability and potential for customization or improvements.

Summary Table:

Cluster - Avg.Price - Bedrooms - Bathrooms - Target Buyers

0 -> 887,388.80 -> 4.25 -> 3.13 -> Affluent buyers

1 -> 493,740.72 -> 3.30 -> 1.96 -> Families valuing lot size

2 -> 574,462.93 -> 3.86 -> 2.21 -> Buyers seeking character

3 -> 459,304.00 -> 3.29 -> 2.14 -> Professionals

4 -> 360,931.50 -> 2.79 -> 1.48 -> Budget-Conscious buyers

Hierarchical Clustering - Hierarchical Clustering is a clustering method that creates a hierarchy of clusters by iteratively merging smaller clusters or splitting larger clusters. The result is typically represented as a dendrogram (a tree-like structure), which shows how clusters are related to each other at different levels of granularity.

	price	bedrooms	bathrooms	living_measure	lot_measure	floors	ceiling_measure	basement	total_area	Age_of_house	Freq
Agglo_Clusters											
0	8.991500e+05	7.125000	7.000000	4144.625000	13875.750000	2.000000	3735.000000	653.750000	17452.000000	46.500000	8
1	8.346156e+05	7.750000	3.750000	3700.468750	8845.375000	1.843750	2646.250000	1042.500000	12755.062500	69.250000	32
2	1.114788e+06	0.000000	0.000000	3649.250000	11962.000000	2.500000	3402.000000	0.000000	14887.750000	24.000000	2
3	4.737202e+05	3.270593	1.913309	1907.708014	8412.300371	1.395345	1645.804106	257.561868	10361.639616	44.973474	19679
4	8.979409e+05	4.295983	3.522727	3583.824260	11724.299683	1.945560	3031.539641	545.468288	15096.523256	25.276427	1892

Cluster Analysis and Recommendations based on the Hierarchical Clustering -

Cluster 0 Analysis

- Key Features:
 - Highest price cluster: \$899,150 on average.
 - Spacious homes: Large living areas (4,144.63 sq. ft.) and lots (13,875.75 sq. ft.).
 - Many bedrooms and bathrooms: 7.13 bedrooms and 7 bathrooms.
 - Younger houses: 46.5 years old on average.
 - Basements: Considerable basement size (653.75 sq. ft.).
 - Frequency: Rare cluster, only 8 houses.
 - Demographic: Likely high-net-worth individuals or families looking for luxurious estates with ample space.
- Recommendations:
 - Target ultra-luxury buyers with premium marketing strategies.
 - Highlight features like large basements, extra rooms, and private outdoor space.
 - Focus on exclusive real estate platforms.

Cluster 1 Analysis

- Key Features:
 - High price: \$834,615 on average.
 - Large homes: 7.75 bedrooms, 3.75 bathrooms, and 3,700.47 sq. ft. living space.
 - Moderate lot size: 8,845.38 sq. ft.
 - Older homes: 69.25 years old on average.
 - Basements present: 1,042.50 sq. ft. on average.
- Frequency: 32 houses.
- Demographic: Families or buyers who value a mix of space and affordability (relative to Cluster 0).
- Recommendations:

- Target upper-middle-class buyers or families who need large homes.
- Promote the balance of space and affordability compared to Cluster 0.
- Highlight potential renovations or upgrades to modernize older homes.

Cluster 2 Analysis

- Key Features:
 - Highest price: 1,114,788 (luxury homes).
 - No bedrooms or bathrooms: Likely large vacant land or unique properties (e.g., commercial spaces).
 - Spacious lots: 11,962 sq. ft. on average.
 - No basements but significant floor and ceiling space.
 - Newer properties: 24 years old on average.
- Frequency: 2 properties (extremely rare).
- Demographic: Ultra-niche buyers, developers, or investors.
- Recommendations:
 - Focus on developers and investors interested in large land parcels.
 - Market these properties for redevelopment or commercial use.
 - Use targeted outreach to specialized real estate investors.

Cluster 3 Analysis

- Key Features:
 - Moderate price: \$473,720 (affordable homes).
 - Average bedrooms and bathrooms: 3.27 and 1.91, respectively.
 - Smaller homes: 1,907.71 sq. ft. living area and 8,412.3 sq. ft. lot.
 - Basements limited: 257.56 sq. ft. on average.
 - Older homes: 44.97 years old.
- Frequency: Dominant cluster with 19,679 houses.

- Demographic: Middle-class families or first-time homebuyers.
- Recommendations:
 - Market homes as affordable, family-friendly options.
 - Highlight practical features like smaller maintenance costs and good location potential.
 - Partner with mortgage brokers to attract first-time buyers.

Cluster 4 Analysis

- Key Features:
 - High price: \$897,941 on average.
 - Spacious homes: 4.30 bedrooms, 3.52 bathrooms, 3,583.82 sq. ft. living space.
 - Large lots: 11,724.30 sq. ft.
 - Moderate basements: 545.47 sq. ft.
 - Older homes: 25.28 years old on average.
- Frequency: 1,892 houses.
- Demographic: Affluent families or professionals seeking high-end homes.
- Recommendations:
 - Market homes as premium family-friendly properties.
 - Highlight features like spacious layouts, modern designs, and outdoor spaces.
 - Focus on suburban luxury buyers.

General Recommendations:

1 - For all clusters:

- Use geospatial data (e.g., region, zip code) to refine marketing strategies.
- Emphasize unique selling points (e.g., spaciousness, affordability, luxury).
- Develop tailored renovation packages for older homes.

2 - Property-specific:

- For rare clusters (0, 2), focus on exclusivity and niche marketing.
- For dominant clusters (3), emphasize affordability and practicality for families.

2 - Predictive Modelling:

Before creating the model, we will split the data into training and testing set. We have split the data into 70:30 ratio.

1 - Linear Regression - Linear Regression is a statistical method used to model the relationship between one or more independent variables (predictors) and a dependent variable (target) by fitting a straight line to the data.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared	
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.93091	101724.620849	0.713786	

This table shows the performance metrics of a Linear Regression model:

- Training Score (0.704): The model explains 70.4% of the variance in the training data.
- Test Score (0.714): The model explains 71.4% of the variance in the test data, indicating good generalization.
- MSE (1.78e+10): Mean Squared Error measures the average squared prediction error.
- RMSE (133,476): Root Mean Squared Error is the square root of MSE, representing the average prediction error in the same units as the target variable.
- MAE (101,725): Mean Absolute Error is the average of absolute prediction errors.
- R-squared (0.714): Proportion of variance explained by the model on the test data. Higher is better, max = 1.

2 - Ridge Regression (Scaling Important) - Ridge Regression is a type of linear regression that adds a penalty term ($\lambda||\beta||_2$) to the loss function to reduce overfitting.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
0	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846

This table summarizes the performance of a Ridge Regression model:

- Training Score (0.704): The model explains 70.4% of variance in training data.
- Test Score (0.714): Explains 71.4% of variance in test data, indicating good generalization.
- MSE (1.78e+10): Mean Squared Error reflects the average squared prediction error.
- RMSE (133,462): Root Mean Squared Error shows the average prediction error in target units.
- MAE (101,712): Mean Absolute Error is the average absolute prediction error.
- R-squared (0.714): Indicates 71.4% of variance in test data is explained by the model.

3. Lasso Regression - A linear regression technique that adds an L1 regularization penalty to the loss function. Purpose of Shrinks some coefficients to zero, performing feature selection and preventing overfitting.

This table summarizes the performance of a Lasso Regression model:

- Training Score (0.704): The model explains 70.4% of variance in training data.
- Test Score (0.714): Explains 71.4% of variance in test data, showing good generalization.
- MSE (1.78e+10): Mean Squared Error represents the average squared prediction error.

- RMSE (133,476): Root Mean Squared Error indicates the average prediction error in target units.
- MAE (101,725): Mean Absolute Error shows the average absolute prediction error.
- R-squared (0.714): Explains 71.4% of the variance in the test data.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
0	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
0	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788

4 - Decision Tree Regression – A non-linear regression technique that splits data into regions by creating a tree-like structure of decisions. Purpose of Predict the target variable by learning simple decision rules from features.

This table summarizes the performance of a Decision Tree Regression model:

- Training Score (0.999): The model almost perfectly explains variance in training data, indicating potential overfitting.
- Test Score (0.498): Explains only 49.8% of variance in test data, showing poor generalization.
- MSE (3.12e+10): High Mean Squared Error indicates large prediction errors on the test set.
- RMSE (176,766): Root Mean Squared Error reflects significant average prediction errors.
- MAE (124,714): Mean Absolute Error indicates large average absolute prediction errors.
- R-squared (0.498): Only 49.8% of the variance in the test data is explained by the model.

This suggests the Decision Tree Regression model is overfitting, performing very well on training data but poorly on the test data.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
0	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
0	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
0	Decision Tree Regression	0.999816	0.498028	3.124604e+10	176765.501982	124714.344540	0.498028

5 - Random Forest Regression: An ensemble method that combines predictions from multiple decision trees (trained on different data subsets and features). Purpose of Improves accuracy and reduces overfitting compared to a single decision tree.

This table summarizes the performance of a Random Forest Regression model:

- Test Score (0.737): Explains 73.7% of the variance in test data, indicating good generalization.
- MSE (1.64e+10): Mean Squared Error is lower compared to other models, indicating better predictions.
- RMSE (127,988): Root Mean Squared Error shows lower average prediction errors.
- MAE (92,837): Mean Absolute Error reflects smaller average absolute prediction errors.
- R-squared (0.737): Explains 73.7% of variance in test data, showing better performance than simpler models.

This indicates Random Forest Regression performs well with balanced accuracy and reduced overfitting compared to the Decision Tree model. It's effective for both linear and non-linear relationships.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
0	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
0	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
0	Decision Tree Regression	0.999816	0.498028	3.124604e+10	176765.501982	124714.344540	0.498028
0	Random Forest Regression	0.962011	0.736836	1.638102e+10	127988.340477	92837.379758	0.736836



6 - Support Vector Regression (SVR) - A regression technique based on Support Vector Machines, which fits a line within a margin of tolerance (ϵ) for the prediction error.

This table summarizes the performance of a Support Vector Regression (SVR) model:

- Training Score (-0.060): The model performs poorly on the training data, explaining less than 0% of the variance, which indicates a poor fit.
- Test Score (-0.053): The model also performs poorly on the test data, showing negative performance, which means it's not better than a simple mean predictor.
- MSE (6.56e+10): Very high Mean Squared Error, indicating large prediction errors.
- RMSE (256,044): Root Mean Squared Error is high, reflecting large average prediction errors in the target variable's units.
- MAE (192,185): Mean Absolute Error shows very high average absolute prediction errors.
- R-squared (-0.053): A negative R-squared suggests the model is worse than simply predicting the mean value for all observations (indicating poor model performance).

This indicates that Support Vector Regression in this case is underperforming and may not be suitable for this dataset. Adjusting hyperparameters or trying a different model might be necessary

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
0	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
0	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
0	Decision Tree Regression	0.999816	0.498028	3.124604e+10	176765.501982	124714.344540	0.498028
0	Random Forest Regression	0.962011	0.736836	1.638102e+10	127988.340477	92837.379758	0.736836
0	Support Vector Regression	-0.059632	-0.053212	6.555876e+10	256044.457442	192185.297520	-0.053212

Model Tuning and Business implication –

7 - Hyperparameter Tuning using Grid Search for Support Vector –

Best Hyperparameters:

- C = 100: Controls the trade-off between achieving a low error on the training data and a smooth decision boundary.
- epsilon = 1: Defines the margin of tolerance for errors.
- kernel = 'linear': Specifies a linear kernel function, meaning the model assumes a linear relationship.
- Training Score (0.698): The model explains 69.8% of variance in the training data.
- Test Score (0.706): Explains 70.6% of variance in the test data, indicating good generalization.
- MSE (1.83e+10): Mean Squared Error is relatively lower than before, suggesting improved prediction accuracy.
- RMSE (135,171): Root Mean Squared Error shows a reasonable average error in predictions.
- MAE (102,065): Mean Absolute Error indicates moderate prediction errors.
- R-squared (0.706): Indicates that 70.6% of the variance in test data is explained by the model.

Best Hyperparameters: {'C': 100, 'epsilon': 1, 'kernel': 'linear'}

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
1	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
2	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
3	Decision Tree Regression	0.999816	0.498028	3.124604e+10	176765.501982	124714.344540	0.498028
4	Random Forest Regression	0.962011	0.736836	1.638102e+10	127988.340477	92837.379758	0.736836
5	Support Vector Regression	-0.059632	-0.053212	6.555876e+10	256044.457442	192185.297520	-0.053212
6	SVR (Best)	0.697728	0.706468	1.827133e+10	135171.467510	102064.899963	0.706468

Conclusion: Hyperparameter tuning improved the SVR model's performance significantly, with a good balance between the training and test scores, suggesting better generalization and lower errors compared to the unoptimized model.

8 - Hyperparameter Tuning using Grid Search for Random Forest –

This table summarizes the performance of the Random Forest Regression model after Hyperparameter Tuning using Grid Search:

Best Hyperparameters:

- max_depth = 30: Maximum depth of the trees, controlling tree complexity.
- min_samples_leaf = 1: Minimum number of samples required in a leaf node.
- min_samples_split = 10: Minimum number of samples required to split an internal node.
- n_estimators = 200: Number of trees in the forest.

Performance Metrics:

- Training Score (0.908): Explains 90.8% of variance in the training data, showing a very good fit.
- Test Score (0.741): Explains 74.1% of variance in the test data, indicating good generalization.
- MSE (1.61e+10): Mean Squared Error is relatively low, reflecting improved prediction accuracy.

- RMSE (126,964): Root Mean Squared Error indicates the average prediction error in the same unit as the target variable.
- MAE (92,561): Mean Absolute Error reflects low average absolute prediction errors.
- R-squared (0.741): Explains 74.1% of variance in the test data.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
1	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
2	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
3	Decision Tree Regression	0.999816	0.492591	3.158442e+10	177720.061533	125278.861043	0.492591
4	Random Forest Regression	0.962085	0.737156	1.636112e+10	127910.592565	93030.416815	0.737156
5	Support Vector Regression	-0.059632	-0.053212	6.555876e+10	256044.457442	192185.297520	-0.053212
6	SVR (Best)	0.697728	0.706468	1.827133e+10	135171.467510	102064.899963	0.706468
7	Random Forest (Best)	0.908053	0.741033	1.611981e+10	126963.832942	92560.785343	0.741033

Conclusion: Hyperparameter tuning improved the Random Forest model's performance. The balance between training and test scores shows the model has reduced overfitting and achieved better generalization, making it robust for predictions.

9 - Hyperparameter Tuning using Grid Search for Decision Tree -

This table summarizes the performance of the Decision Tree Regression model after Hyperparameter Tuning using Grid Search:

Best Hyperparameters:

- max_depth = 10: Limits the maximum depth of the tree to prevent overfitting.
- max_features = None: Considers all features when splitting a node.
- min_samples_leaf = 4: Minimum number of samples required in a leaf node.
- min_samples_split = 10: Minimum number of samples required to split an internal node.

Performance Metrics:

- Training Score (0.760): Explains 75.9% of variance in training data, showing a decent fit.

- Test Score (0.685): Explains 68.5% of variance in test data, indicating reasonable generalization.
- MSE (1.96e+10): Mean Squared Error indicates moderate average prediction errors.
- RMSE (139,939): Root Mean Squared Error reflects the average error in the same unit as the target variable.
- MAE (102,910): Mean Absolute Error indicates average absolute prediction errors.
- R-squared (0.685): Explains 68.5% of variance in the test data.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
1	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
2	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
3	Decision Tree Regression	0.999816	0.492591	3.158442e+10	177720.061533	125278.861043	0.492591
4	Random Forest Regression	0.962085	0.737156	1.636112e+10	127910.592565	93030.416815	0.737156
5	Support Vector Regression	-0.059632	-0.053212	6.555876e+10	256044.457442	192185.297520	-0.053212
6	SVR (Best)	0.697728	0.706468	1.827133e+10	135171.467510	102064.899963	0.706468
7	Random Forest (Best)	0.908053	0.741033	1.611981e+10	126963.832942	92560.785343	0.741033
8	Decision Tree (Best)	0.759647	0.685396	1.958301e+10	139939.294875	102910.296355	0.685396

Conclusion: Hyperparameter tuning improved the Decision Tree model, balancing its performance between training and test datasets. While it performs better than the original untuned version, it still has slightly lower accuracy compared to ensemble methods like Random Forest. It's suitable for simpler and interpretable models.

10 - Define Hyperparameter Grids Search for Ridge –

Best Hyperparameter:

- alpha = 1: Controls the regularization strength. A higher value increases regularization, reducing overfitting.

Performance Metrics:

- Training Score (0.704): Explains 70.4% of variance in training data, indicating a good fit.

- Test Score (0.714): Explains 71.4% of variance in test data, showing good generalization.
- MSE (1.78e+10): Mean Squared Error reflects moderate average squared prediction errors.
- RMSE (133,462): Root Mean Squared Error indicates the average prediction error in target units.
- MAE (101,712): Mean Absolute Error reflects the average absolute prediction error.
- R-squared (0.714): Explains 71.4% of variance in the test data.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
1	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
2	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
3	Decision Tree Regression	0.999816	0.492591	3.158442e+10	177720.061533	125278.861043	0.492591
4	Random Forest Regression	0.962085	0.737156	1.636112e+10	127910.592565	93030.416815	0.737156
5	Support Vector Regression	-0.059632	-0.053212	6.555876e+10	256044.457442	192185.297520	-0.053212
6	SVR (Best)	0.697728	0.706468	1.827133e+10	135171.467510	102064.899963	0.706468
7	Random Forest (Best)	0.908053	0.741033	1.611981e+10	126963.832942	92560.785343	0.741033
8	Decision Tree (Best)	0.759647	0.685396	1.958301e+10	139939.294875	102910.296355	0.685396
9	Ridge (CV)	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846

Conclusion: Hyperparameter tuning for Ridge Regression optimized its performance, achieving a balanced model with good generalization. This suggests the Ridge model effectively handles multicollinearity and prevents overfitting while maintaining strong prediction accuracy.

11 - Define Hyperparameter Grid Search for Lasso -

This table summarizes the performance of the Lasso Regression model after Hyperparameter Tuning using Grid Search:

Best Hyperparameter:

alpha = 1: Controls the regularization strength for L1 penalty. Higher values increase regularization, shrinking coefficients and performing feature selection.

Performance Metrics:

- Training Score (0.704): Explains 70.4% of variance in training data, indicating a good fit.
- Test Score (0.714): Explains 71.4% of variance in test data, showing good generalization.
- MSE (1.78e+10): Mean Squared Error reflects moderate average squared prediction errors.
- RMSE (133,476): Root Mean Squared Error shows the average prediction error in target units.
- MAE (101,725): Mean Absolute Error reflects the average absolute prediction error.
- R-squared (0.714): Indicates the model explains 71.4% of the variance in the test data.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
1	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
2	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
3	Decision Tree Regression	0.999816	0.492591	3.158442e+10	177720.061533	125278.861043	0.492591
4	Random Forest Regression	0.962085	0.737156	1.636112e+10	127910.592565	93030.416815	0.737156
5	Support Vector Regression	-0.059632	-0.053212	6.555876e+10	256044.457442	192185.297520	-0.053212
6	SVR (Best)	0.697728	0.706468	1.827133e+10	135171.467510	102064.899963	0.706468
7	Random Forest (Best)	0.908053	0.741033	1.611981e+10	126963.832942	92560.785343	0.741033
8	Decision Tree (Best)	0.759647	0.685396	1.958301e+10	139939.294875	102910.296355	0.685396
9	Ridge (CV)	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
10	Lasso (CV)	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788

Conclusion:

Hyperparameter tuning for Lasso Regression optimized its performance, balancing accuracy and regularization. This indicates the model is effective for reducing overfitting and selecting the most relevant features while maintaining strong prediction accuracy.

Ensemble Methods

12 - Gradient Boosting - This table summarizes the performance of the Gradient Boosting Regression model after Hyperparameter Tuning:

Best Hyperparameters:

- `learning_rate = 0.1`: Controls the contribution of each tree to the final model, balancing learning speed and accuracy.
- `max_depth = 5`: Limits the maximum depth of each tree to reduce overfitting.
- `n_estimators = 300`: Number of boosting stages (trees) used in the model.
- `subsample = 0.8`: Fraction of samples used for training each tree, helping to reduce overfitting.

Performance Metrics:

- Training Score (0.841): Explains 84.1% of variance in training data, indicating a good fit.
- Test Score (0.749): Explains 74.9% of variance in test data, showing strong generalization.
- MSE (1.56e+10): Mean Squared Error is lower compared to other models, reflecting better accuracy.
- RMSE (125,051): Root Mean Squared Error indicates the average prediction error in target units.
- MAE (92,212): Mean Absolute Error reflects lower average absolute prediction errors.
- R-squared (0.749): Explains 74.9% of the variance in test data, making it one of the best-performing models.

Conclusion:

The Gradient Boosting model performs well, with strong accuracy on both training and test data. Hyperparameter tuning further improved the model's performance by balancing training and generalization, making it suitable for handling complex relationships in the data.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
1	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
2	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
3	Decision Tree Regression	0.999816	0.492591	3.158442e+10	177720.061533	125278.861043	0.492591
4	Random Forest Regression	0.962085	0.737156	1.636112e+10	127910.592565	93030.416815	0.737156
5	Support Vector Regression	-0.059632	-0.053212	6.555876e+10	256044.457442	192185.297520	-0.053212
6	SVR (Best)	0.697728	0.706468	1.827133e+10	135171.467510	102064.899963	0.706468
7	Random Forest (Best)	0.908053	0.741033	1.611981e+10	126963.832942	92560.785343	0.741033
8	Decision Tree (Best)	0.759647	0.685396	1.958301e+10	139939.294875	102910.296355	0.685396
9	Ridge (CV)	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
10	Lasso (CV)	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
11	Gradient Boosting	0.841255	0.748777	1.563775e+10	125051.003897	92211.757602	0.748777

13 – XGBoost Regression Model after Hyperparameter Tuning – This table summarizes the performance of the XGBoost Regression model:

Performance Metrics:

- Training Score (0.785): Explains 78.5% of variance in the training data, indicating a good fit without overfitting.
- Test Score (0.746): Explains 74.6% of variance in the test data, showing strong generalization.
- MSE (1.58e+10): Mean Squared Error indicates good prediction accuracy.
- RMSE (125,745): Root Mean Squared Error reflects moderate average prediction errors in target units.
- MAE (93,236): Mean Absolute Error shows reasonably low average absolute prediction errors.
- R-squared (0.746): Indicates that 74.6% of the variance in test data is explained by the model.

Conclusion: The XGBoost model performs well, with accuracy comparable to Gradient Boosting. Its slightly lower training score suggests better resistance

to overfitting while maintaining strong predictive power. It's efficient and well-suited for capturing non-linear relationships in the data.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.704312	0.713786	1.781582e+10	133475.930910	101724.620849	0.713786
1	Ridge Regression	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
2	Lasso Regression	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
3	Decision Tree Regression	0.999816	0.492591	3.158442e+10	177720.061533	125278.861043	0.492591
4	Random Forest Regression	0.962085	0.737156	1.636112e+10	127910.592565	93030.416815	0.737156
5	Support Vector Regression	-0.059632	-0.053212	6.555876e+10	256044.457442	192185.297520	-0.053212
6	SVR (Best)	0.697728	0.706468	1.827133e+10	135171.467510	102064.899963	0.706468
7	Random Forest (Best)	0.908053	0.741033	1.611981e+10	126963.832942	92560.785343	0.741033
8	Decision Tree (Best)	0.759647	0.685396	1.958301e+10	139939.294875	102910.296355	0.685396
9	Ridge (CV)	0.704310	0.713846	1.781210e+10	133461.989347	101712.419205	0.713846
10	Lasso (CV)	0.704309	0.713788	1.781573e+10	133475.569240	101724.728485	0.713788
11	Gradient Boosting	0.841255	0.748777	1.563775e+10	125051.003897	92211.757602	0.748777
12	XGBoost	0.784564	0.745982	1.581177e+10	125744.853207	93235.754258	0.745982

14 – AdaBoost Regression model after Hyperparameter Tuning - This table summarizes the performance of the AdaBoost Regression model after Hyperparameter Tuning.

Best Hyperparameters:

- learning_rate = 0.01: Controls the contribution of each weak learner to the final model, balancing learning speed and accuracy.
- n_estimators = 200: Number of weak learners (trees) in the ensemble.

Performance Metrics:

- Training Score (0.613): Explains 61.3% of variance in the training data, indicating a moderate fit.
- Test Score (0.622): Explains 62.2% of variance in the test data, showing reasonable generalization but lower accuracy compared to other models.
- MSE (2.35e+10): Mean Squared Error indicates higher average prediction errors compared to Gradient Boosting and XGBoost.

- RMSE (153,418): Root Mean Squared Error shows relatively higher prediction errors in target units.
- MAE (121,480): Mean Absolute Error reflects larger average absolute prediction errors.
- R-squared (0.622): Explains 62.2% of variance in test data, indicating moderate model performance.

Conclusion:

The AdaBoost model performs reasonably well but is outperformed by models like Gradient Boosting and XGBoost. While it's effective for handling simpler patterns, it may struggle with more complex relationships. Adjusting the weak learner (base estimator) or further tuning could potentially improve its performance.

Metrics Explanation:

1 - Training Score (R^2): Indicates how well the model explains variance in the training data. Higher is better, but excessively high scores (near 1.0) may indicate overfitting.

2 - Test Score (R^2): Indicates how well the model generalizes to unseen data. Higher is better.

3 - Mean Squared Error (MSE): Measures average squared errors; lower values are better.

4 - Root Mean Squared Error (RMSE): Square root of MSE, providing error in original units; lower values are better.

5 - Mean Absolute Error (MAE): Average absolute error in prediction; lower values are better.

6 - R-squared (R^2): Proportion of variance explained by the model; higher values are better.

Comparison of Models:

1. Overfitting Check -

- Decision Tree Regression: Training Score = 0.999816 (almost 1), while Test Score = 0.500042. This suggests overfitting.
- Random Forest Regression: Training Score = 0.961738 and Test Score = 0.737817. Shows good generalization with minimal overfitting.

2. Best Generalization (Test R^2): The highest Test R^2 is from Gradient Boosting Regression (0.746752), indicating this model generalizes better than the others.

3. Error Analysis: Models with lower RMSE, MAE, and MSE indicate better performance. The lowest errors are observed in Gradient Boosting and Random Forest Regression:

- Gradient Boosting:
 - RMSE = 125,554
 - MAE = 92,414
- Random Forest:
 - RMSE = 126,964
 - MAE = 92,561

4. SVR and AdaBoost: Both models underperform in terms of R^2 (close to 0 or negative for some configurations) and have significantly higher RMSE/MAE values.

5. Linear Models (Ridge/Lasso): Perform decently with consistent R^2 (~0.713846) and moderate errors but are outperformed by tree-based models.

Conclusion:

Based on these insights, **Gradient Boosting Regression** is the best model overall:

- Highest Test R^2 (0.746752).
- Lowest RMSE (125,554).
- Consistent and interpretable results without signs of overfitting.

Perform Principal Component Analysis -

PCA reduces dimensionality by transforming features into a smaller number of uncorrelated components that explain the maximum variance in the data.

When to use: Useful when features are highly correlated or when the number of features is large compared to the number of samples.

```
Number of components explaining 95% variance: 29
Explained variance ratio: [0.1328031  0.07230026 0.05706973 0.04310868 0.03934352 0.03635796
 0.0343309  0.03005875 0.0284128  0.02786743 0.02715681 0.02695691
 0.02624506 0.02615837 0.02571487 0.02540947 0.02535183 0.02507943
 0.02487063 0.02462585 0.0243145  0.02421338 0.02398924 0.02370432
 0.02285191 0.02246216 0.01988402 0.01632685 0.01527238]
Total variance explained: 0.95
```

After Performing the PCA we have again run the all model on with useful feature.

1 - Linear Regression -

- Training Score (0.687): The model now explains 68.7% of the variance in the training data, showing a slight reduction.
- Test Score (0.698): Explains 69.8% of the variance in the test data, slightly lower than before.
- MSE (0.3008): Mean Squared Error is significantly smaller, but this is likely due to rescaling of the data during PCA.
- RMSE (0.5485): Root Mean Squared Error is also much smaller, reflecting rescaled target units.
- MAE (0.4176): Average absolute error is much smaller, again likely due to the new scale.
- R-squared (0.698): Explains 69.8% of variance in the test data, only slightly lower than before PCA.

Conclusion - After applying PCA -

- The test score (R-squared) decreased slightly, indicating a small loss in predictive power.
- However, PCA likely reduced multicollinearity, making the model simpler and possibly more stable.
- The scaling introduced by PCA changed the MSE, RMSE, and MAE metrics, so comparisons on those metrics should consider this adjustment.
- PCA helps focus on the most relevant features, but it may lead to a slight trade-off in accuracy, as seen here.

	Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
0	Linear Regression	0.68701	0.697858	0.300816	0.548467	0.417612	0.697858

2 - Ridge Regression –

Performance Metrics:

- Training Score (0.687): Explains 68.7% of the variance in the training data, slightly lower due to reduced feature complexity.
- Test Score (0.698): Explains 69.8% of the variance in the test data, with a small decline.
- MSE (0.3008): Reflects rescaled features and target, no longer directly comparable to pre-PCA values.
- RMSE (0.5485): Much smaller due to scaling introduced by PCA.
- MAE (0.4176): Lower average absolute error due to rescaled target.
- R-squared (0.698): Slight reduction in explained variance compared to pre-PCA.

Key Observations:

1. Accuracy: Both training and test scores slightly decreased after PCA, as feature reduction often simplifies the model at the cost of some variance explanation.
2. Multicollinearity: PCA removed multicollinearity by transforming features into uncorrelated principal components, reducing the need for Ridge regularization.
3. Error Metrics: Post-PCA error metrics (MSE, RMSE, MAE) appear smaller due to feature and target scaling. They cannot be directly compared to pre-PCA values.
4. Model Complexity: PCA reduced the dimensionality of the dataset, simplifying the model while maintaining reasonable accuracy.

Conclusion:

Applying PCA to Ridge Regression led to slightly reduced accuracy but significantly simplified the model by focusing on key components. This trade-off is ideal for scenarios where computational efficiency and reduced feature

complexity are prioritized. However, for retaining maximum predictive power, the pre-PCA Ridge Regression model performs better.

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Linear Regression	0.687010	0.697858	0.300816	0.548467	0.417612	0.697858
Ridge Regression	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850

3 - Lasso Regression –

Performance Metrics:

- Training Score (0.687): Explains 68.7% of the variance in the training data, slightly lower due to reduced feature complexity.
- Test Score (0.698): Explains 69.8% of the variance in the test data, with a small decline.
- MSE (0.3008): Reflects rescaled features and target, no longer directly comparable to pre-PCA values.
- RMSE (0.5485): Much smaller due to scaling introduced by PCA.
- MAE (0.4176): Lower average absolute error due to rescaled target.
- R-squared (0.698): Slight reduction in explained variance compared to pre-PCA.

Key Observations:

1 – Accuracy - Both training and test scores slightly decreased after PCA, as feature reduction often simplifies the model at the cost of some variance explanation.

2 – Multicollinearity - PCA removed multicollinearity by transforming features into uncorrelated principal components, reducing the need for Ridge regularization.

3 - Error Metrics - Post-PCA error metrics (MSE, RMSE, MAE) appear smaller due to feature and target scaling. They cannot be directly compared to pre-PCA values.

4 - Model Complexity: PCA reduced the dimensionality of the dataset, simplifying the model while maintaining reasonable accuracy.

Conclusion:

Applying PCA to Ridge Regression led to slightly reduced accuracy but significantly simplified the model by focusing on key components. This trade-off

is ideal for scenarios where computational efficiency and reduced feature complexity are prioritized. However, for retaining maximum predictive power, the

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Linear Regression	0.687010	0.697858	0.300816	0.548467	0.417612	0.697858
Ridge Regression	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso Regression	0.532644	0.538576	0.459398	0.677789	0.532276	0.538576

pre-PCA Ridge Regression model performs better.

4 - Decision Tree Regression –

Performance Metrics:

- Training Score (0.9998): Still a near-perfect fit, suggesting the Decision Tree is fully memorizing the training data, even with reduced feature space.
- Test Score (0.4261): Explains only 42.6% of the variance in the test data, a drop compared to pre-PCA.
- MSE (0.5714): Lower mean squared error, reflecting rescaled features and target due to PCA.
- RMSE (0.7559): Smaller root mean squared error because of the normalized scale.
- MAE (0.5298): Reduced average absolute error, again due to rescaled target values.
- R-squared (0.4261): Further decline in the ability to explain variance in the test data.

Key Observations:

1 - Overfitting: The Training Score remains perfect (0.9998) in both cases, indicating the Decision Tree still overfits to the training data, even after PCA.

2 - Reduced Generalization: The Test Score (R-squared) decreased from 0.493 to 0.426, suggesting that PCA reduced the model's ability to generalize to unseen data.

3 - Impact of PCA: PCA likely removed important features that contributed to variance explanation, further weakening the Decision Tree's test performance.

4 - Error Metrics: Post-PCA metrics like MSE, RMSE, and MAE appear smaller but are rescaled, making them non-comparable to pre-PCA values.

Conclusion:

Applying PCA to the Decision Tree Regression worsened its generalization performance (Test Score dropped to 42.6%). This suggests that PCA removed some crucial features, and the Decision Tree's inherent overfitting behavior was not mitigated. The pre-PCA model, despite being overfitted, explained more variance in the test data and performed better in predicting the target.

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Linear Regression	0.687010	0.697858	0.300816	0.548467	0.417612	0.697858
Ridge Regression	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso Regression	0.532644	0.538576	0.459398	0.677789	0.532276	0.538576
Decision Tree Regression	0.999816	0.426097	0.571382	0.755898	0.529788	0.426097

5 - Random Forest Regression –

Performance Metrics:

- Training Score (0.961): Slight decrease in the training score, still explaining 96.1% of variance, showing consistent model fit.
- Test Score (0.726): Explains 72.6% of the variance in the test data, slightly reduced generalization.
- MSE (0.2728): Lower due to feature and target normalization by PCA; not directly comparable to pre-PCA.
- RMSE (0.5223): Reduced due to PCA-induced scaling.
- MAE (0.3819): Lower average absolute error due to rescaling.
- R-squared (0.726): Slight reduction from pre-PCA (73.7% to 72.6%).

Key Observations:

1. Accuracy: Both training and test scores dropped slightly after PCA, but the difference is minimal, indicating PCA did not significantly affect Random Forest performance.
2. Error Metrics: Post-PCA error metrics (MSE, RMSE, MAE) are much smaller due to scaling introduced by PCA, making them non-comparable to pre-PCA metrics.
3. Generalization: A small drop in R-squared (from 0.737 to 0.726) indicates a slight decrease in variance explanation for test data after PCA.

4. Feature Dimensionality: PCA might have removed less relevant features while retaining most of the important ones, leading to minimal accuracy changes.

Conclusion:

The Random Forest model was slightly impacted by PCA, with minor reductions in both training and test scores. While PCA helped simplify the dataset, it did not provide a significant advantage for Random Forest, which is inherently robust to multicollinearity and feature scaling. Pre-PCA Random Forest results offer slightly better generalization and are preferable for this dataset. For further improvement, hyperparameter tuning or increasing ensemble depth might yield better results.

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Linear Regression	0.687010	0.697858	0.300816	0.548467	0.417612	0.697858
Ridge Regression	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso Regression	0.532644	0.538576	0.459398	0.677789	0.532276	0.538576
Decision Tree Regression	0.999816	0.426097	0.571382	0.755898	0.529788	0.426097
Random Forest Regression	0.960928	0.725993	0.272803	0.522306	0.381948	0.725993

6 - Support Vector Regression -

Performance Metrics:

- Training Score (0.5421): A significant improvement, indicating the model now explains 54.2% of the variance in the training data.
- Test Score (0.5493): The model generalizes better, explaining 54.9% of the variance in the test data.
- MSE (0.4487): Lower due to normalization by PCA, but not directly comparable with pre-PCA results.
- RMSE (0.6698): Lower because of PCA scaling.
- MAE (0.5052): Reduced due to feature scaling.
- R-squared (0.5493): A significant improvement, indicating the model now explains 54.9% of the variance in the test data.

Key Observations:

1. Performance Improvement: PCA drastically improved the performance of SVR. The negative R-squared pre-PCA indicates poor fitting and

generalization, while post-PCA results show moderate success in explaining data variance.

2. Error Metrics: Post-PCA metrics (MSE, RMSE, MAE) are smaller due to PCA's scaling effects, making them non-comparable to pre-PCA values.
3. Generalization: Post-PCA, the R-squared improved from -0.0532 to 0.5493, demonstrating the model's improved ability to predict test data.
4. Role of PCA: PCA likely reduced irrelevant or noisy features and improved the numerical stability of SVR, which is sensitive to feature scaling and dimensionality.

Conclusion - Support Vector Regression (SVR) benefited significantly from PCA. While pre-PCA SVR struggled to fit the data, PCA helped the model achieve moderate predictive accuracy. Post-PCA SVR is a much better fit for this dataset, although it still lags behind other models like Random Forest or Gradient Boosting in terms of overall performance.

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Linear Regression	0.687010	0.697858	0.300816	0.548467	0.417612	0.697858
Ridge Regression	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso Regression	0.532644	0.538576	0.459398	0.677789	0.532276	0.538576
Decision Tree Regression	0.999816	0.426097	0.571382	0.755898	0.529788	0.426097
Random Forest Regression	0.960928	0.725993	0.272803	0.522306	0.381948	0.725993
Support Vector Regression	0.542052	0.549347	0.448674	0.669832	0.505169	0.549347

7 - Define SVR model and parameter grid for GridSearchCV

Model	Training Score	Test Score (R ²)	MSE	RMSE	MAE
Before PCA (SVR, Best Params: {'C': 100, 'epsilon': 1, 'kernel': 'linear'})	0.6977	0.7065	1.83×10^{10}	1.83×10^5	135,171.47
			1.83×10^{10}		102,064.90

Model	Training Score	Test Score (R^2)	MSE	RMSE	MAE
After PCA (SVR, Best Params: {'kernel': 'rbf', 'epsilon': 0.1, 'C': 10})	0.5690	0.5725	0.4257	0.6524	0.4902

Observations

1. R-squared:

- Before PCA: 0.7065
- After PCA: 0.5725
Performance dropped significantly after PCA, indicating information loss.

2. Hyperparameter Shift:

- Before PCA: Linear kernel, C=100, epsilon=1 → Stronger regularization.
- After PCA: RBF kernel, C=10, epsilon=0.1 → Weaker regularization and non-linearity.

3. Training Score Drop:

- Before PCA: 0.6977
- After PCA: 0.5690
- Suggests the PCA transformation reduced model complexity, leading to underfitting.

4. Error Metrics:

- Before PCA: MSE = 1.83×10^{10} , RMSE = 135,171.47, MAE = 102,064.90
- After PCA: MSE = 0.4257, RMSE = 0.6524, MAE = 0.4902
- The drastic scale difference suggests that PCA significantly altered the data distribution.

Conclusion - Best pre-PCA model (linear kernel, C=100, epsilon=1) performed better than post-PCA SVR.

8 - Hyperparameter Tuning using Grid Search for Random Forest –

Performance Metrics:

- Training Score: 0.945915 (higher)
- Test Score: 0.731755 (slightly lower)
- MSE: 0.267067 (scaled down due to feature transformation)
- RMSE: 0.516786
- MAE: 0.378456
- R^2 Score: 0.731755 (slightly lower)

Observations:

1. Training Score Increased:

- After PCA, the model fits the training data better (0.908 → 0.945), meaning it captures more patterns.
- This could indicate overfitting—it performs too well on training data but slightly worse on test data.

2. Test Score Slightly Dropped:

- Pre-PCA R^2 : 0.7410 → Post-PCA R^2 : 0.7317
- PCA removed some features, possibly reducing performance slightly.

3. MSE/RMSE Scaled Down in Post-PCA:

- PCA scales down feature magnitudes, which affects error values.
- RMSE went from 126,963 (pre-PCA) to 0.516 (post-PCA), likely due to rescaled data.

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Random Forest (Best)	0.945915	0.731755	0.267067	0.516786	0.378456	0.731755

9 - Hyperparameter Tuning using Grid Search for Decision Tree –

Performance Metrics:

- Training Score: 0.7725 (slightly better)
- Test Score: 0.6161 (lower)
- MSE: 0.3822 (scaled due to PCA)
- RMSE: 0.6182
- MAE: 0.4488
- R^2 Score: 0.6161 (dropped slightly)

Observations:

✅ Hyperparameter tuning improved training score (0.7596 → 0.7725), meaning the model fits training data better.

❌ Test score dropped (0.6854 → 0.6161), indicating some loss in predictive power.

📉 MSE & RMSE scaled down due to PCA, but this does not mean actual error reduced—it's just a scaling effect.

Conclusion –

- Hyperparameter tuning improved training accuracy (0.7596 → 0.7725), showing better learning.
- Test score dropped (0.6854 → 0.6161), indicating a slight loss in predictive performance.
- The MSE & RMSE decreased due to feature scaling, but this doesn't directly mean better predictions.

10 - Define Hyperparameter Grids Search for Ridge –

Model	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Ridge (Before PCA)	0.704310	0.713846	1.78e+10	133,462	101,712	0.713846
Ridge (After PCA)	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850

Key Takeaways:

1- Slight Drop in Performance – Test score dropped from 0.7138 to 0.6978, indicating a slight loss of predictive power.

2 - Lower MSE, RMSE, and MAE – The error metrics have improved significantly, but this could be due to PCA scaling the data differently.

3 - PCA May Have Removed Useful Features – Ridge Regression tends to work well with correlated features, and PCA reduces dimensionality, which might have removed valuable information.

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Linear Regression	0.687010	0.697858	0.300816	0.548467	0.417612	0.697858
Ridge Regression	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso Regression	0.532644	0.538576	0.459398	0.677789	0.532276	0.538576
Decision Tree Regression	0.999816	0.426097	0.571382	0.755898	0.529788	0.426097
Random Forest Regression	0.960928	0.725993	0.272803	0.522306	0.381948	0.725993
Support Vector Regression	0.542052	0.549347	0.448674	0.669832	0.505169	0.549347
Ridge (CV)	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850

11 - Define Hyperparameter Grids Search for Lasso -

Performance Metrics

Model	Training Score	Test Score (R ²)	MSE	RMSE	MAE
Before PCA (Lasso)	0.7043	0.7138	1.78×10^{10}	133,475.57	101,724.73
After PCA (Lasso)	0.6866	0.6972	0.3015	0.5491	0.4184

Observations

1. R-squared: Slightly decreased from 0.7138 to 0.6972 after PCA, indicating a minor loss in explanatory power.
2. Error Metrics: Significant scale difference in MSE, RMSE, and MAE post-PCA, suggesting data standardization or transformation effects.

3. Training Score Drop: The training score reduced from 0.7043 to 0.6866, implying some loss of model fit after dimensionality reduction.
4. Best Lasso Parameter: The best alpha value found was 0.001, indicating minimal regularization was needed.

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Linear Regression	0.687010	0.697858	0.300816	0.548467	0.417612	0.697858
Ridge Regression	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso Regression	0.532644	0.538576	0.459398	0.677789	0.532276	0.538576
Decision Tree Regression	0.999816	0.426097	0.571382	0.755898	0.529788	0.426097
Random Forest Regression	0.960928	0.725993	0.272803	0.522306	0.381948	0.725993
Support Vector Regression	0.542052	0.549347	0.448674	0.669832	0.505169	0.549347
Ridge (CV)	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso (CV)	0.686571	0.697191	0.301479	0.549071	0.418360	0.697191

12 - Gradient Boosting –

Performance Metrics:

- Training Score: 0.89002
- Test Score: 0.74063
- MSE: 0.258231
- RMSE: 0.508164
- MAE: 0.373159
- R^2 Score: 0.74063

Observations

1. R-squared:
 - Before PCA: 0.7488
 - After PCA: 0.7406
 - Performance slightly declined post-PCA, suggesting some information loss.
2. Training Score Increased:

- Before PCA: 0.8413
- After PCA: 0.8900
- The increase indicates potential overfitting, as the gap between training and test scores widened.

3. Hyperparameter Adjustments:

- Max Depth: Increased from 5 to 7, making trees deeper and possibly overfitting.
- Learning Rate: Lowered from 0.1 to 0.05, allowing for more gradual learning.
- n_estimators: Increased from 300 to 400, increasing model complexity.
- Subsample Reduced (0.8 → 0.6), min_samples_split (10) & min_samples_leaf (4) introduced for regularization.
- max_features set to 'log2', which may help prevent overfitting.

4. Error Metrics:

- Post-PCA MSE, RMSE, and MAE have significantly changed scale, likely due to data transformations.
- Lower MAE (0.3732) post-PCA suggests better predictions in the transformed space.

Conclusion

✅ Gradient Boosting remains one of the best-performing models with a high R^2 .

⚠️ Overfitting risk increased post-PCA due to a wider training-test score gap.

📉 Performance slightly dropped after PCA, meaning important variance may have been lost.

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Gradient Boosting	0.89002	0.74063	0.258231	0.508164	0.373159	0.74063

13 – XGBoost –

Performance Metrics

Model	Training Score	Test Score (R^2)	MSE	RMSE	MAE
Before PCA (XGBoost)	0.7846	0.7460	1.58×10^{10}	125,744.85	93,235.75
After PCA (XGBoost)	0.8111	0.7378	0.2611	0.5110	0.3801

Observations

1. R-squared: The test score dropped slightly from 0.7460 to 0.7378 after PCA, showing minor performance reduction.
2. Training Score Increased: The training score improved from 0.7846 to 0.8111, suggesting a better fit on training data post-PCA.
3. Error Metrics:
 - RMSE and MAE changed scale significantly, indicating a potential effect of data transformation.
 - MSE reduced drastically post-PCA, but this might be due to differences in feature scaling.
4. Overfitting Risk: The training score increased while the test score decreased slightly, which could indicate overfitting after PCA.

Conclusion - XGBoost performed better than Lasso in both cases, achieving a higher R^2 .

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Linear Regression	0.687010	0.697858	0.300816	0.548467	0.417612	0.697858
Ridge Regression	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso Regression	0.532644	0.538576	0.459398	0.677789	0.532276	0.538576
Decision Tree Regression	0.999816	0.426097	0.571382	0.755898	0.529788	0.426097
Random Forest Regression	0.960928	0.725993	0.272803	0.522306	0.381948	0.725993
Support Vector Regression	0.542052	0.549347	0.448674	0.669832	0.505169	0.549347
Ridge (CV)	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso (CV)	0.686571	0.697191	0.301479	0.549071	0.418360	0.697191
XGBoost	0.811090	0.737767	0.261081	0.510961	0.380103	0.737767

14 – AdaBoost –

Performance Metrics -

Model	Training Score	Test Score (R^2)	MSE	RMSE	MAE
AdaBoost	0.5667	0.5652	0.4329	0.6579	0.5368

Observations

1. R-squared: The test R^2 is 0.5652, which is relatively lower than XGBoost and Lasso, suggesting that AdaBoost is not capturing as much variance in the data.
2. Training Score: The training score is quite close to the test score (0.5667 vs. 0.5652), indicating that there's not a significant overfitting problem here.
3. Error Metrics:
 - MSE and RMSE suggest moderate error levels.
 - MAE is relatively high at 0.5368, indicating that, on average, the predictions are off by around 0.54 units, which could still be improved.
4. Best AdaBoost Parameters: The best parameters are `learning_rate = 0.1` and `n_estimators = 100`, which appear to be reasonable for the dataset.

Conclusion

- AdaBoost has a lower performance compared to XGBoost and Lasso based on R^2 , suggesting it might not be the best model for this dataset.

Model_Name	Training Score	Test Score	MSE	RMSE	MAE	R-squared
Linear Regression	0.687010	0.697858	0.300816	0.548467	0.417612	0.697858
Ridge Regression	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso Regression	0.532644	0.538576	0.459398	0.677789	0.532276	0.538576
Decision Tree Regression	0.999816	0.426097	0.571382	0.755898	0.529788	0.426097
Random Forest Regression	0.960928	0.725993	0.272803	0.522306	0.381948	0.725993
Support Vector Regression	0.542052	0.549347	0.448674	0.669832	0.505169	0.549347
Ridge (CV)	0.687009	0.697850	0.300823	0.548474	0.417624	0.697850
Lasso (CV)	0.686571	0.697191	0.301479	0.549071	0.418360	0.697191
XGBoost	0.811090	0.737767	0.261081	0.510961	0.380103	0.737767
AdaBoost	0.566685	0.565235	0.432856	0.657918	0.536765	0.565235